

درس سیستم عامل

تمرین عملی هفتم سوال 1

402105727

متین باقری

402105802

فاطمیما تیمارچی

سوال 1

مطابق عکس های زیر تست های usertests و cowtests همگی pass شدند :

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
```

```
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openiput: OK
test exitiput: OK
```

```
xv6-riscv: make — Konsole
New Tab Split View
Copy Paste Find...
usertrap(): sepc=0x45e0 stval=0x22014000
usertrap(): unexpected scause 0xf pid=7828
sepc=0x45e0 stval=0x23014000
usertrap(): unexpected scause 0xf pid=7829
sepc=0x45e0 stval=0x24014000
usertrap(): unexpected scause 0xf pid=7830
sepc=0x45e0 stval=0x25014000
usertrap(): unexpected scause 0xf pid=7831
sepc=0x45e0 stval=0x26014000
usertrap(): unexpected scause 0xf pid=7832
sepc=0x45e0 stval=0x27014000
usertrap(): unexpected scause 0xf pid=7833
sepc=0x45e0 stval=0x28014000
usertrap(): unexpected scause 0xf pid=7834
sepc=0x45e0 stval=0x29014000
usertrap(): unexpected scause 0xf pid=7835
sepc=0x45e0 stval=0x2a014000
usertrap(): unexpected scause 0xf pid=7836
sepc=0x45e0 stval=0x2b014000
usertrap(): unexpected scause 0xf pid=7837
sepc=0x45e0 stval=0x2c014000
usertrap(): unexpected scause 0xf pid=7838
sepc=0x45e0 stval=0x2d014000
usertrap(): unexpected scause 0xf pid=7839
sepc=0x45e0 stval=0x2e014000
usertrap(): unexpected scause 0xf pid=7840
sepc=0x45e0 stval=0x2f014000
usertrap(): unexpected scause 0xf pid=7841
sepc=0x45e0 stval=0x30014000
usertrap(): unexpected scause 0xf pid=7842
sepc=0x45e0 stval=0x31014000
usertrap(): unexpected scause 0xf pid=7843
sepc=0x45e0 stval=0x32014000
usertrap(): unexpected scause 0xf pid=7844
sepc=0x45e0 stval=0x33014000
usertrap(): unexpected scause 0xf pid=7845
sepc=0x45e0 stval=0x34014000
usertrap(): unexpected scause 0xf pid=7846
sepc=0x45e0 stval=0x35014000
usertrap(): unexpected scause 0xf pid=7847
sepc=0x45e0 stval=0x36014000
usertrap(): unexpected scause 0xf pid=7848
sepc=0x45e0 stval=0x37014000
usertrap(): unexpected scause 0xf pid=7849
sepc=0x45e0 stval=0x38014000
usertrap(): unexpected scause 0xf pid=7850
sepc=0x45e0 stval=0x39014000
usertrap(): unexpected scause 0xf pid=7851
sepc=0x45e0 stval=0x3a014000
usertrap(): unexpected scause 0xf pid=7852
sepc=0x45e0 stval=0x3b014000
usertrap(): unexpected scause 0xf pid=7853
sepc=0x45e0 stval=0x3c014000
usertrap(): unexpected scause 0xf pid=7854
sepc=0x45e0 stval=0x3d014000
usertrap(): unexpected scause 0xf pid=7855
sepc=0x45e0 stval=0x3e014000
usertrap(): unexpected scause 0xf pid=7856
sepc=0x45e0 stval=0x3f014000
OK
test lazy_copy: OK
ALL TESTS PASSED
$
```

برای اینکار فایل های زیر رو تغییر دادم :

: Kernel/riscv.h -1

در ابتدا با استفاده از خط زیر یک بیت جدید برای تشخیص صفحات COW تعریف کردم . در این صورت اگه PTW_W=0 و PTE_COW=0 باشه صفحه ما read-only هست اما اگه PTW_W=0 و PTE_COW=1 باشه اون وقت صفحه COW هست و در صورت نوشتن باید کپی بشه

```
#define PTE_COW (1L << 8)
```

: Kernel/vm.c -2

uvmcopy (A

تابع uvmcopy رو به صورت زیر تغییر دادم چون روش قبلی کند بود و حافظه رو الکی هدر میداد . در copy-one-write هنگام fork دیگه هیچ صفحه ای واقعا کپی نمیشه و والد و فرزند به همون صفحه فیزیکی اشاره میکنن و فقط وقتی که یکی از پدر یا فرزند بخواد بنویسه اون موقع فقط کپی انجام میشه به عبارت دیگه والد صفحه فیزیکی کپی نمیشه و همون page به فرزند map میشه حالا اگه صفحه writeable بود اون وقت PTE_W پاک میشه و PTE_COW ست میشه و شمارنده ارجاع صفحه زیاد میشه .

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            continue;
        if((*pte & PTE_V) == 0)
            continue;

        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if(flags & PTE_W){
            *pte = (*pte & ~PTE_W) | PTE_COW;
```

```

    flags = (flags & ~PTE_W) | PTE_COW;
}

if(mappages(new, i, PGSIZE, pa, flags) != 0)
    goto err;

    kincrcf((void*)pa);
}

sfence_vma();
return 0;

err:
uvmunmap(new, 0, i / PGSIZE, 1);
return -1;
}

```

cowfault (B)

تابع cowfault رو به صورت زیر نوشتیم . دلیل اضافه کردن این تابع این هست که وقتی پردازش روی صفحه COW مینویسه چون PTE_W برابر صفر هست اون وقت سخت افزار خاکی store page fault میده و کنترل وارد کرنل میشه و کرنل هست که تصمیم میگیره آیا این صفحه باید کپی بشه یا نه . بنابراین اول بررسی میکنیم که آیا COW هست یا نه اگر نبود که میفهمیم ایم خطای یک خطای واقعی هست . در گام بعد تعداد ارجاعات رو بررسی میکنیم اگر یه پردازش از این صفحه استفاده میکرد که لازم نیست اونو کپی کنیم و تنها کافیست که PTE_W رو فعال کنیم و PTE_COW رو پاک کنیم . اما اگر تعداد ارجاعات بیشتر از یکی بود اون وقت باید یک صفحه جدید تخصیص بدیم و محتوای صفحه قبل رو کپی کنیم و سپس mapping پردازش به صفحه جدید تغییر بدیم و شمارنده از جاع صفحه قدیمی رو کاهش بدیم و در نهایت صفحه جدید writable میشه بنابراین پردازش میتونه بدون هیچ مشکلی بنویسه و سایر پردازش ها همچنان نسخه قدیمی رو دارند .

```

int
cowfault(pagetable_t pagetable, uint64 va)
{
    pte_t *pte;
    uint64 pa;
    uint flags;

```

```

char *mem;

va = PGROUNDDOWN(va);
if(va >= MAXVA)
    return -1;

pte = walk(pagetable, va, 0);
if(pte == 0)
    return -1;
if((*pte & PTE_V) == 0)
    return -1;
if((*pte & PTE_U) == 0)
    return -1;
if((*pte & PTE_COW) == 0)
    return -1;

pa = PTE2PA(*pte);
flags = PTE_FLAGS(*pte);

if(krefcount((void*)pa) == 1){
    flags = (flags | PTE_W) & ~PTE_COW;
    *pte = PA2PTE(pa) | flags;
    sfence_vma();
    return 0;
}

mem = kalloc();
if(mem == 0)
    return -1;

memmove(mem, (char*)pa, PGSIZE);

kfree((void*)pa);

flags = (flags | PTE_W) & ~PTE_COW;
*pte = PA2PTE((uint64)mem) | flags;

```

```
sfence_vma();
return 0;
}
```

copyout (C

این تابع رو به صورت زیر تغییر دادم . چون در این تابع داده از کرنل به حافظه user نوشته میشه حالا اگه این نوشتن روی صفحه COW باشه اون وقت چون PTE_W برابر صفر هست بنابراین کرنل نمیتونست چیزی بنویسه بنابراین تابع رو تغییر دادم . به این صورت که اگه PTE_W=0 و PTE_COW=1 بود اون وقت تابع cowfault رو صدا میزنم تا بتونه بنویسه . در آخر هم با استفاده از walk آدرس جدید رو میگیریم چون ممکنه cowfault صفحه فیزیکی رو عوض کرده باشه پس آدرس فیزیکی دیگه معتبر نیست و باید آدرس جدید رو بگیریم.

```
if((*pte & PTE_W) == 0){
    if((*pte & PTE_COW) && ((*pte & PTE_U) != 0)){
        if(cowfault(pageable, va0) != 0)
            return -1;
        pte = walk(pageable, va0, 0);
        if(pte == 0 || (*pte & PTE_W) == 0)
            return -1;

        pa0 = walkaddr(pageable, va0);
        if(pa0 == 0)
            return -1;
    } else {
        return -1;
    }
}
```

: Kernel/kalloc.c -3

در COW ممکنه یک صفحه فیزیکی همزمان توسط چند تا پردازش به اشتراک گذاشته بشه در این حالت چون نمیتونیم وقتی هر پردازش unmap کرد مستقیم صفحه رو آزاد کنیم برای همین به ازای هر صفحه یک شمارنده (refcnt) نگه میدارم . و سپس با استفاده از pai2idx آدرس فیزیکی رو به اندیس آرایه تبدیل میکنم :

```
#define NPHYPAGES ((PHYSTOP - KERNBASE) / PGSIZE)
static int refcnt[NPHYPAGES];

static inline int
pa2idx(uint64 pa)
{
    return (pa - KERNBASE) / PGSIZE;
}
```

سپس تابع kincrcf رو به صورت زیر اضافه کردم . علت اضافه شدن این تابع هم این هست که در for خب ما مطابق تابع uvmcopy می آمدیم یک صفحه رو به child هم map میکردیم پس در نتیجه اون صفحه باید یک رفرنس جدید بگیره پس refcount باید با یک جمع بشه . همچنین از kmem.lock هم استفاده کردم چون ممکنه چند پردازش همزمان بتونن refcount رو تغییر بدن .

```
void
kincrcf(void *pa)
{
    uint64 x = (uint64)pa;
    if(x < KERNBASE || x >= PHYSTOP || (x % PGSIZE) != 0)
        panic("kincrcf");

    acquire(&kmem.lock);
    refcnt[pa2idx(x)]++;
    release(&kmem.lock);
}
```

سپس تابع krefcount رو به صورت زیر اضافه کردم . علت اضافه شدن این تابع هم این هست که تو تابع cowfault وقتی خطای writefault رخ میداد(این خطا زمانی رخ میده که یک صفحه فیزیکی توسط چند پردازنده به اشتراک گذاشته بشه و یکی از پردازش ها بخواد روی صفحه چیزی بنویسه) اگه فقط یک ارجاع داشت که نمیخواست کپی کنیم و با writeable تنها کار درمیومد اما اگه چند تا ارجاع داشتیم اون وقت باید کپی میکردیم پس در این تابع باید refcount رو میخواندیم و مثل قبل باید lock هم میکردیم .(refcount برای این هست که کرنل در COW بدونه یک صفحه با چند نفر به اشتراک گذاشته شده)

```
int
krefcount(void *pa)
{
    uint64 x = (uint64)pa;
    if(x < KERNBASE || x >= PHYSTOP || (x % PGSIZE) != 0)
        return 0;
}
```

```

    acquire(&kmem.lock);
    int r = refcnt[pa2idx(x)];
    release(&kmem.lock);
    return r;
}

```

تابع freerange رو برای هماهنگی با kfree جدید به صورت زیر تغییر دادم. در حالت جدی برخلاف حالت قبلی دیگه kfree همیشه صفحه رو آزاد نمیکند بلکه اول میاد refcnt رو منهای یک میکنه اگه refcnt مقدار صفر داشت اون وقت kfree پانیک میکنه یا اینکه صفحه هیچوقت آزاد نمیشه بنابراین من قبل از kfree(p) مقدار refcnt صفحه رو یک میکنم تا kfree اون رو به صفر برسونه تا صفحه رو آزاد کنه.

```

void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE){
        acquire(&kmem.lock);
        refcnt[pa2idx((uint64)p)] = 1;
        release(&kmem.lock);
        kfree(p);
    }
}

```

تابع kfree رو به صورت زیر تغییر دادم. در حالت قبل kfree همیشه صفحه رو وارد freelist میکرد اما در حالت جدید چون در COW ممکنه یک پردازنده دیگه از صفحه ما استفاده کنه نمیتونیم مستقیم این صفحه رو آزاد کنیم بلکه باید تنها refcnt رو کم کنیم و وقتی refcnt صفر شد در اون حالت صفحه رو آزاد کنیم.

```

void
kfree(void *pa)
{
    struct run *r;
    uint64 x = (uint64)pa;
    if((x % PGSIZE) != 0 || (char*)pa < end || x >= PHYSTOP)
        panic("kfree");
    acquire(&kmem.lock);
    int idx = pa2idx(x);
    if(refcnt[idx] < 1)
        panic("kfree: refcnt");
}

```

```

    refcnt[idx]--;
    if(refcnt[idx] > 0){
        release(&kmem.lock);
        return;
    }
    release(&kmem.lock);
    memset(pa, 1, PGSIZE);
    r = (struct run*)pa;
    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}

```

در انتها کم kalloc رو به صورت زیر اصلاح کردم تا در لحظه ای که یک صفحه از freelist برداشته میشه یعنی الان این صفحه ما یک ارجاع داره پس باید مقدار refcnt آن برابر 1 شود.

```

refcnt[pa2idx((uint64)r)] = 1;

```

Kernel/trap.c -4

چون COW باعث ایجاد خطا میشد بنابراین در usertrap رو به صورت زیر تغییر دادم . وقتی که scause برابر

15 شد اون وقت آدرس خطا رو از stval میگیرم سپس بررسی میکنم که آیا صفحه مربوطه در PTE دارای PTE_COW هست یا نه اگر بود که اون وقت تابع cowfault رو صدا میزنم . اگر هم نبود اون وقت میفهمم این خطا واقعا خطای واقعی هست و میزیم سراغ vmfault و اگر هم هیچ کدوم نتونستن خطا رو اوکی کنن پردازنده kill میشه . علت اینکه از vmfault تنها هم استفاده نکردم این بود که در wmfault اگر صفحه map نشده باشه اون وقت اونو allocate میکنه اما در COW صفحه map شده اما writeable نیست پس باید یک مکانیزم جدا باید برایش در نظر میگرفتم .

```

} else if((which_dev = devintr()) != 0){
    // ok
    } else if(r_scause() == 15 && cowfault(p->pagetable,
r_stval()) == 0) {

    } else if((r_scause() == 15 || r_scause() == 13) &&

```



```

vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0)
!= 0) {
    // page fault on lazily-allocated page
} else {

    printf("usertrap(): unexpected scause 0x%lx pid=%d\n",
r_scause(), p->pid);
    printf("          sepc=0x%lx stval=0x%lx\n", r_sepc(),
r_stval());
    setkilled(p);
}

```

Kernel/defs.h -5

در اینجا prototype های زیر رو اضافه کردم تا فایل های کرنل بتونن بدون خطای کمپایل از اونا استفاده کنن .

```

void      kincref(void *);
int       krefcount(void *);

```

```

// vm.c
int       cowfault(pagetable_t, uint64);

```

کارکرد کلی :

در ابتدا به وسیله بیت نرم افزاری PTE_COW در ورودی های جدول صفحات اومدم مکانیزم copy-on-write رو پیاده سازی کردم . در اینجا بعد از زدن fork در تابع uvmcopy به جای کپی فیزیکی صفحات والد و فرزند میایم از یک صفحه فیزیکی مشترک استفاده میکنیم . بنابراین برای فعال شدن COW مجوز نوشتن PTE_W از هر دو تا PTE برداشتم و بیت PTE_COW فعال میشه . حالا اگه بخوایم روی این صفحات بویسیم خطای stor page fault رخ میده و در نتیجه کنترل به کرنل منتقل میشه . بنابراین ما میایم با استفاده از تابع cowfault تعداد ارجاع ها به صفحه مورد نظر رو پیدا میکنیم اگه تعداد ارجاع ها که همون refcount هست مساوی یک باشه صفحه ما writable میشه و دیگه لازم نیست کپی کنیم اما اگه refcount بزرگتر از یک بشه یک صفحه جدید اختصاص داده میشه و داده ها کپی میشه و صفحه جدید دیگه writable میشه . در انتها هم برای free کردن صفحات با استفاده از توابع kalloc.c میایم بررسی میکنیم که تنها صفحه در حالتی آزاد بشه که آخرین ارجاع بهش حذف شده باشه . در انتها هم کرنل از طریق تابع copyout میفهمه که صفحه ما COW هست یا نه

اگه COW نبود که به صورت عادی کارش رو ادامه میده اما اگه COW بود که مکانیزم های مخصوص COW رو انجام میده .